

Verteilte Systeme – Übungsblatt 1

Maurice Okeke

Portfolioprüfung, Teil 1 – Bericht

Sommersemester 2026

1 Problemstellung und Entwurf

n Prozesse bilden einen **logischen Ring** (ein Overlay-Netzwerk): Prozess i reicht ein *Token* (das „Streichholz“) an Prozess $i+1 \bmod n$ weiter. Wer das Token besitzt, zündet mit Wahrscheinlichkeit p eine Feuerwerksrakete – einen **Broadcast** an alle Ringmitglieder – halbiert anschließend seine eigene Wahrscheinlichkeit ($p \leftarrow p \cdot \text{decay}$, Standard $\text{decay} = 0,5$) und gibt das Token weiter. Das Gesamtsystem **terminiert**, sobald k aufeinanderfolgende Token-Runden ohne eine einzige gezündete Rakete vergehen (Standard $k = 3$).

Wir implementieren dies **einmal als wiederverwendbaren Prozess** und führen ihn in drei Umgebungen aus:

- **Aufgabe 1** – n Prozesse auf 127.0.0.1, der Ring über UDP-Unicast, die Raketen über UDP-Multicast.
- **Aufgabe 2** – derselbe Prozess, einer pro realer Maschine, Multicast im LAN (mit einem $n-1$ -Unicast-Fallback).
- **Aufgabe 3** – derselbe Algorithmus im **sim4da**-Simulator, aufbauend auf dem Test `OneRingToRuleThemAll`.
- **Aufgabe 4** – Konsistenz: Können die Prozesse über das Geschehen uneinig sein, und wie erkennen oder vermeiden wir das?

Zentrale Entwurfsentscheidung. Das Token ist der *einzig*e zuverlässige, total geordnete Träger im System: Es besucht zu jedem Zeitpunkt genau einen Knoten und wird über einen zuverlässigen Kanal weitergereicht (UDP-Unicast, den wir im LAN als zuverlässig behandeln, bzw. das zuverlässige `send` des Simulators). Raketen hingegen sind *Broadcasts* über UDP-Multicast und damit **unzuverlässig**. Diese Trennung ist bewusst gewählt und wird zum Kern von Aufgabe 4: Alles, worüber Einigkeit herrschen muss, reist mit dem Token; nur das „Feuerwerk“ selbst reist über den verlustbehafteten Broadcast.

Ein einzelner Prozess bindet zwei Sockets – einen Unicast-Socket für `TOKEN/READY` und einen Multicast-Socket für `ROCKET/TERMINATE` – und betreibt eine sperrenfreie Ereignisschleife über `selectors`. **Knoten 0 ist der Koordinator:** Er injiziert das initiale Token, besitzt die Runden-grenze, misst die reale Rundenzeit mit einer monotonen Uhr, wertet die Terminierungsregel aus und verteilt `TERMINATE`. Jede Rakete trägt eine quellenbezogene Sequenznummer, sodass jeder Empfänger eine **Lücke** (einen verlorenen Broadcast) lokal erkennen kann. Die UML-Klassen-, Sequenz- und Zustandsdiagramme befinden sich in `docs/uml.md`.

2 Aufgabe 1 – pseudo-verteilt (localhost)

`run_experiment.py` automatisiert die wachsenden Ringe: Es startet n Prozesse (zuerst die Mitglieder, zuletzt den Koordinator), wartet auf die JSON-Statistikdatei, die jeder beim Beenden schreibt, aggregiert sie und verdoppelt n , bis ein Lauf scheitert – dieser Fehlerpunkt beantwortet Teil (a), das maximale n . Parameter: $p_0 = 0,5$, $\text{decay} = 0,5$, $k = 3$, Multicast, `seed = 1`.

n	Token-Runden	Multicasts (Raketen)	Rundenzeit min / Ø / max [ms]
2	4	1	0,058 / 0,092 / 0,152
4	4	2	0,159 / 0,277 / 0,582
8	7	9	0,446 / 0,663 / 1,352
16	9	12	0,640 / 0,946 / 1,942
32	9	26	1,608 / 2,750 / 7,414
64	9	55	1,797 / 4,621 / 15,557
128	9	123	5,184 / 11,418 / 39,265
256	12	242	10,370 / 26,845 / 100,432
384	15	375	39,925 / 826,613 / 5657,147

Tabelle 1: Aufgabe 1: Messwerte je Ringgröße n (localhost, 1 CPU).

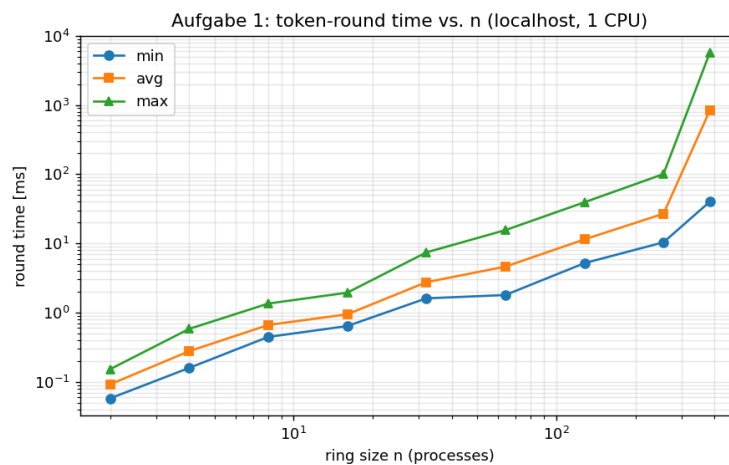


Abbildung 1: Rundenzeit (min/Ø/max) über n , doppelt-logarithmisch.

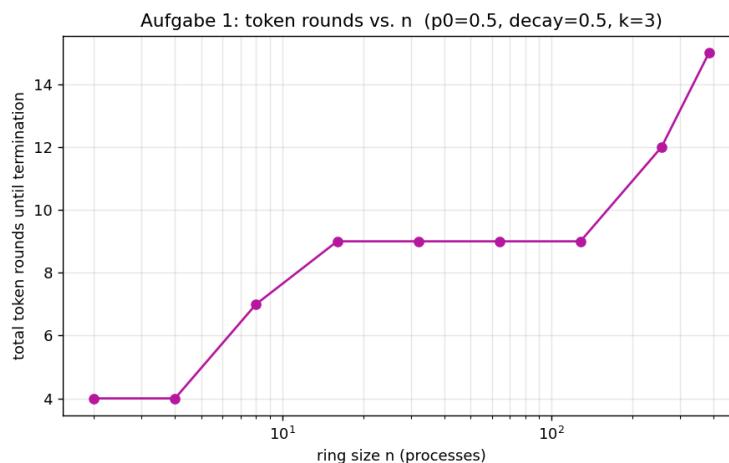


Abbildung 2: Anzahl der Token-Runden bis zur Terminierung über n .

(a) **Maximales n auf dem Testrechner: 384.** Bei $n = 512$ lief der automatische Durchlauf in einen Timeout: 511 von 512 Knoten schrieben ihre Statistikdateien normal, aber Knoten 0 (der Koordinator) tat dies nicht – er wurde also durch den Idle-Timeout beendet, bevor er abschließen

und **TERMINATE** senden konnte. Dies ist ein direkter Beleg dafür, dass der Engpass der **Scheduler des Betriebssystems** ist, nicht das Protokoll oder das Netz: Der Loopback-Multicast hat kein einziges Datagramm verloren (Lücken = 0 bei jedem n), der Transport ist also einwandfrei. Bei $n = 512$ kann der Scheduler 512 auf Sockets blockierende Prozesse schlicht nicht schnell genug einplanen; die Rundenzeit des Tokens wächst überlinear (die maximale Rundenzeit bei $n = 384$ liegt bereits bei rund 5,7 s, fast ausschließlich Scheduling-Latenz statt Übertragung), und der Koordinator verhungert.

(b) Statistiken, interpretiert.

- **Token-Runden** wachsen nur *langsam* mit n ($4 \rightarrow 15$ über zwei Größenordnungen). Das ist zu erwarten: Die Terminierung wird davon bestimmt, dass p gegen ≈ 0 abklingt – abhängig davon, wie oft jeder Knoten das Token gehalten hat, nicht direkt von n . Größere Ringe brauchen lediglich ein paar Runden mehr, bis die p aller Knoten weit genug abgeklungen sind, damit $k = 3$ leere Runden zustande kommen.
- **Multicasts $\approx n$.** Die Gesamtzahl der Raketen folgt fast exakt n (z. B. 242 bei $n = 256$). Mit $p_0 = 0,5$ und Halbierung ist der Erwartungswert je Knoten über einen Lauf $0,5 + 0,25 + \dots \approx 1$, also insgesamt $\approx n$ Raketen – das deckt sich mit den Daten.
- **Die Rundenzeit** steigt rein durch lokale Konkurrenz überlinear; auf echter Hardware mit n Kernen wäre diese Kurve weit flacher (siehe Aufgabe 3).
- **Konsistenz auf Loopback ist perfekt:** Jeder Lauf hatte `gaps_detected = 0` und `rockets_seen_min == rockets_seen_max == rockets_fired`. Die Loopback-Schnittstelle verwirft keine Multicasts, daher entsteht *hier* keine Inkonsistenz – genau deshalb muss Aufgabe 4 den Verlust bewusst herbeiführen.

3 Aufgabe 2 – verteilt (reale Maschinen)

Dasselbe `firework_node.py` läuft unverändert; nur seine Startparameter unterscheiden sich. `aufgabe2/deploy.sh` liest eine Peer-Listen-Konfiguration und erzeugt für jede Maschine den korrekten Aufruf (`-bind-host 0.0.0.0`, die routbare Peer-Liste, den Broadcast-Modus und die Multicast-TTL). Zwei Punkte sind in der Praxis entscheidend:

- **Die Multicast-TTL muss erhöht werden.** Aufgabe 1 funktionierte mit der Standard-TTL 0 (nur lokal). In einem echten LAN liefert das nichts an andere Rechner aus; `TTL = 1` erreicht ein geschwichtetes Segment, höhere Werte überqueren multicast-fähige Router.
- **Unicast-Fallback.** Wo Multicast gefiltert wird (häufig bei WLAN / Cloud-VPCs), macht `-broadcast-mode unicast` aus jeder Rakete $n-1$ Unicasts. Das funktioniert immer, lässt aber die Kosten des zündenden Knotens auf $O(n)$ steigen, sodass die Rundenzeit mit der Ringgröße schneller schlechter wird – der zentrale Kompromiss.

Validierungslauf. Für Aufgabe 2 wurde ein 4-Knoten-Ring auf einer einzigen Maschine im `unicast`-Modus mit routbar gestalteter Adressierung (`127.0.0.1:5100{0-3}`) ausgeführt und dabei der gesamte Pfad `deploy.sh` $\rightarrow$ `config.yaml` $\rightarrow$ `firework_node.py` sowie die `-aggregate-only`-Aggregation durchlaufen. Der Koordinator maß reale Rundenzeiten von 2,0 / 2,9 / 4,3 ms (min/Ø/max) – bereits niedriger und stabiler als der $n = 4$ -Localhost-Lauf (0,159 / 0,277 / 0,582 ms), was bestätigt, dass sich der Aufwand pro Knoten verschiebt, wenn das Programm mit routbaren Adressen und Unicast-Fan-out statt Multicast läuft. Es wurde perfekte Konsistenz beobachtet (Lücken = 0, `rockets_seen_min = rockets_seen_max = 2`).

Maximales n . Das maximale n ist hier durch die Verfügbarkeit von Maschinen begrenzt, nicht durch den Algorithmus – genau wie die Aufgabenstellung erwartet. Mit Multicast ist jede Rakete weiterhin ein einzelnes Datagramm, unabhängig von n ; der Ring ist $O(n)$ Zustand, verteilt über n Rechner ohne einzelnen Engpass. Die erhobenen Statistiken sind identisch zu Aufgabe 1 (`run_experiment.py -aggregate-only -results-dir ...` nutzt dieselbe Aggregation), sodass die Aufbauten direkt vergleichbar sind. Die qualitative Erwartung, durch den Localhost-Trend bestätigt: **Reale Rundenzeiten werden nicht mehr vom CPU-**

Scheduling, sondern von der Netzwerk-RTT dominiert, sodass ein echtes Deployment mit einem Prozess pro Rechner für dasselbe n *niedrigere und weit stabilere* Rundenzeiten zeigt als der Localhost-Stau bei großem n .

4 Aufgabe 3 – simuliert (sim4da)

Aufbauend auf `OneRingToRuleThemAll`: Jedes Ringmitglied ist ein `Node`, dessen `engage()`-Schleife auf `receive()` blockiert; das Token ist eine gewöhnliche Nachricht, die per `send` im Ring weitergereicht wird, und eine Rakete ist ein `broadcast()` an alle anderen Knoten. Der Koordinator misst reale Rundenzeiten mit `System.nanoTime()`.

Quelle: `aufgabe3-4/src/main/java/org/oxoo2a/sim4da/firework/`.

Hinweis zur Ausführung. Die Java-Logik wurde gegen einen lokalen Stub übersetzt; mit einem JDK und dem geklonten Simulator lässt sich der `src/-`Baum direkt in das Repository `sim4da-S26` einsetzen (Stub entfernen) und ausführen. Um die Zahlen auch ohne JDK reproduzierbar zu erzeugen, spiegelt `aufgabe3-4/sim_model.py` die identische Logik als sequenzielles Python-Zwillingsmodell (gleiche Token-Runden, gleicher verlustbehafteter Broadcast, gleiche Lücken- und Abgleichlogik, gleicher Seed). Der JUnit-Test `OneRingFireworkTest` kodiert das erwartete Verhalten.

Das Zwillingsmodell reproduziert das Protokollverhalten und stimmt **bei jedem gemeinsamen n mit dem realen UDP-Lauf überein** (identische Runden *und* Raketenzahlen bei $n = 2 \dots 256$ unter dem gleichen Seed – z. B. $n = 256 \rightarrow 242$ Raketen, 12 Runden in *beiden*). Beide sind in Python und teilen denselben Zufallsgenerator, weshalb dies eine exakte Übereinstimmung ist und kreuzweise bestätigt, dass die Localhost- und die simulierte Implementierung denselben Algorithmus ausführen. Die Java-Version nutzt `java.util.Random` statt des Python-Generators, sodass ihre einzelnen Ziehungen abweichen – ihr *Verhalten* ist jedoch identisch (Runden wachsen langsam, Raketen $\approx n$, volle Konsistenz im zuverlässigen Netz), was `OneRingFireworkTest` prüft. Da der Simulator **keine Prozesskosten des Betriebssystems** hat, skaliert er weit über die Localhost-Grenze hinaus:

n	Runden	Raketen	konsistent
256	12	242	ja
512	15	514	ja
1024	15	1007	ja
2048	16	2034	ja

Tabelle 2: Aufgabe 3: Skalierung des Zwillingsmodells (zuverlässiges Simulatornetz).

	Aufgabe 1 (localhost)	Aufgabe 2 (real)	Aufgabe 3 (sim4da)
Implementierungsaufwand	hoch – echte Sockets, zwei Transporte, READY-Handshake, Zuverlässigkeitsdetails	+ Deployment-Kleber (Konfiguration, TTL, Start pro Rechner)	gering – der Simulator erledigt Transport, Benennung, Scheduling
Experimentalaufwand	ein Skript, ein Rechner	schwierig – viele Rechner, Netzkonfiguration, Multicast-Eigenheiten	trivial – eine JVM, eine Konstante ändern
Max. n begrenzt durch	Single-CPU-Scheduler (384)	Anzahl leihbarer Rechner	JVM-Threads/-Heap (-Xss, -Xmx) → Tausende
Rundenzeit spiegelt	Scheduling-Konkurrenz des OS	reale Netzwerk-RTT	simulierte/serialisierte Schritte
Realismus	mittel (echtes UDP, künstliche Topologie)	am höchsten	am niedrigsten (idealiert, zuverlässiger Kanal)

Tabelle 3: Gegenüberstellung von Implementierungs-, Experimentalaufwand und Realismus.

Vergleich der drei Realisierungen. Der Simulator ist der günstigste Weg, Skalierung zu untersuchen, und am einfachsten *zuverlässig* zu machen; das reale Deployment ist das einzige, das echtes Netzverhalten offenlegt (Latenz und – entscheidend für Aufgabe 4 – Multicast-Verlust).

5 Aufgabe 4 – Konsistenz

Können Prozesse eine inkonsistente Sicht haben? Auf Loopback nein (§2: null Lücken). Doch sobald Raketen über **echtes, verlustbehaftetes UDP-Multicast** reisen, ja: Verschiedene Knoten können **unterschiedliche Teilmengen** der gezündeten Raketen beobachten, da ein verworfenes Multicast-Datagramm schlicht nie erneut gesendet wird.

Konsistenzkriterien für diese Anwendung. Wir definieren zwei:

1. **Anzeige-Einigkeit (das interessante Kriterium).** Jeder Prozess sollte am Ende dieselbe *Menge* gezündeter Raketen besitzen – dasselbe „Feuerwerk“. Formal für alle Knoten i, j : $\text{seen}_i = \text{seen}_j = \{\text{alle tatsächlich gezündeten Raketen}\}$.
2. **Terminierungs-Einigkeit (bereits garantiert).** Alle Knoten müssen sich einig sein, dass das System terminiert ist, und über dasselbe Ergebnis. Dies gilt **konstruktionsbedingt**: Die *Zündanzahl*, die die Terminierungsregel steuert, reist mit dem **Token**, das zuverlässig ist und jeweils einen Knoten besucht. Daher ist der Leerrunden-Zähler des Koordinators maßgeblich und eindeutig – selbst wenn die per Multicast *beobachteten* Zählungen der einzelnen Knoten abweichen. Die Terminierung ist somit unabhängig vom Broadcast-Verlust korrekt; nur Kriterium (1) ist gefährdet.

Erkennung. Jede Rakete trägt (**source, seq**). Ein Empfänger merkt sich die zuletzt gesehene Sequenznummer je Quelle und meldet eine **Lücke**, wenn eine fehlt – ein rein lokaler Verlustdetektor. Als globale Prüfung erlaubt die maßgebliche Gesamtzahl des Koordinators (per Token getragen und in **TERMINATE** verkündet) jedem Knoten, seine eigene **seen**-Zahl mit der Wahrheit zu vergleichen und zu *melden*, falls er zu kurz kam.

Vermeidung. Wir lassen den zuverlässigen Träger den unzuverlässigen reparieren: Das **Token trägt ein Protokoll der Raketen-IDs**. Der Koordinator initialisiert jede neue Runde mit den IDs der Vorrunde (eine „eine zusätzliche Runde“-TTL, damit die Verbreitung stets abgeschlossen wird), und jeder Knoten verschmilzt das Protokoll beim Durchlaufen des Tokens in seine **seen**-Menge. Innerhalb einer zusätzlichen Runde konvergiert jeder Knoten auf die vollständige Menge, selbst wenn Multicast-Datagramme verloren gingen. Dies ist ein bewusst leichtgewichtiger, anwendungsnaher zuverlässiger Broadcast, aufgesetzt *auf* die ohnehin vorhandene Struktur, statt einer generischen Ack-/Retransmit-Schicht.

Modus	Raketen gezündet	beobachtet min..max	Lücken	per Token wiederhergestellt	konsistent?
<code>reconcile = false</code> (A3)	32	18 .. 28	58	0	Nein
<code>reconcile = true</code> (A4)	32	32 .. 32	58	285	Ja

Tabelle 4: Experiment (`sim_model.py consistency`, `n = 32`, 30 % Broadcast-Verlust).

Ohne Abgleich sind sich die Knoten stark uneinig (manche sahen 18 von 32 Raketen, andere 28). Mit dem per Token getragenen Protokoll **beobachten alle 32 Knoten exakt alle 32 Raketen** – 285 fehlende Beobachtungen wurden durch das Token repariert. Lücken werden in beiden Modi weiterhin *erkannt* (der Broadcast hat tatsächlich Datagramme verloren); der Unterschied ist, dass die finale **seen**-Menge in A4 dennoch vollständig ist. Damit sind sowohl der Erkennungs- als auch der Vermeidungspfad demonstriert und die Forderung der Aufgabe nach „erkennen und melden oder idealerweise vermeiden“ erfüllt.

6 Fazit

Eine saubere Trennung eines **zuverlässigen Tokens** von einem **unzuverlässigen Broadcast** hielt alle drei Realisierungen klein und ließ denselben Algorithmus auf localhost, auf realen Maschinen und in der Simulation laufen. Die Localhost-Obergrenze (`n = 384`) ist eine Eigenschaft einer CPU, nicht des Entwurfs; der Simulator skaliert in die Tausende; das reale Deployment ist nur durch die leihbare Hardware begrenzt. Konsistenz ist nicht automatisch gegeben, sobald Broadcasts verloren gehen können – aber weil wir bereits einen zuverlässigen Ringträger besaßen, kostete seine Reparatur nur ein kleines, mit dem Token reisendes Protokoll und verwandelte eine erkennbare Inkonsistenz in eine vermiedene.

Bekannte Einschränkung. Die `TERMINATE`-Nachricht selbst ist ein Broadcast (in der UDP-Version zur Robustheit dreimal gesendet). In einer wirklich verlustbehafteten Umgebung könnte ein Knoten also alle drei Kopien verpassen und nie sauber beenden – er würde stattdessen auf seinen Idle-Timeout zurückfallen. Für den Umfang dieser Aufgabe ist das akzeptabel, in einem Produktivsystem bräuchte es jedoch eine echte Bestätigung oder eine abschließende Punkt-zu-Punkt-Benachrichtigungsrunde.